

Software Requirements Specification (SRS)

ReadLoop - Community Book Sharing Platform

Version: 1.0

Date: September 1, 2025

Prepared by: Development Team

Project: ReadLoop Platform

Table of Contents

1. [Introduction](#)
2. [Overall Description](#)
3. [System Features](#)
4. [External Interface Requirements](#)
5. [Non-Functional Requirements](#)
6. [Other Requirements](#)

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document describes the functional and non-functional requirements for the ReadLoop platform - a community-driven book sharing and marketplace application. The system enables users to share, request, and trade books within a local community network while promoting sustainable reading practices through an intelligent recommendation system.

1.2 Document Conventions

- **High Priority Requirements:** Critical for system functionality

- **Medium Priority Requirements:** Important for user experience
- **Low Priority Requirements:** Nice-to-have features for future releases
- **User Types:** Reader, Book Owner, Community Member, Administrator

1.3 Intended Audience and Reading Suggestions

This document is intended for:

- Software developers and architects
- Project managers and stakeholders
- Quality assurance teams
- System administrators
- User experience designers

1.4 Product Scope

ReadLoop is a web application built with Angular frontend and Node.js backend that facilitates book sharing within local communities. The platform connects book owners with readers, enabling them to share, request, and trade books without the need for purchasing. The system promotes community engagement, sustainable reading practices, and local book circulation through intelligent matching algorithms.

1.5 References

- IEEE Std 830-1998 - IEEE Recommended Practice for Software Requirements Specifications
- Web Content Accessibility Guidelines (WCAG) 2.1
- General Data Protection Regulation (GDPR) compliance standards
- Angular Framework Documentation
- Node.js Best Practices Guide
- Neon Database Documentation

2. Overall Description

2.1 Product Perspective

ReadLoop is a standalone web application that serves as a community marketplace for book sharing. The system operates independently but integrates with external services for enhanced functionality such as book information databases and mapping services.

System Architecture Overview:

- **Frontend:** Angular 17+ with TypeScript
- **Backend:** Node.js with Express.js framework
- **Database:** Neon (Serverless PostgreSQL)
- **Version Control:** GitHub with automated CI/CD
- **External APIs:** Book information services, geolocation services
- **File Storage:** Cloud storage for book images and user photos
- **Deployment:** Cloud-based with GitHub Actions integration

2.2 Product Functions

The major functions of ReadLoop include:

Core Functions:

- User registration and profile management
- Book listing and catalog management
- Book sharing and requesting system
- Community matching and discovery
- Communication system between users
- **AI-Powered Book Recommendation Engine** (Novel Feature)

Supporting Functions:

- Search and filtering capabilities
- User rating and review system
- Notification system
- Community formation and management
- Book condition tracking

- Geographic proximity matching

2.3 User Classes and Characteristics

2.3.1 Primary Users

User Type	Characteristics	Technical Expertise	Usage Frequency
Book Enthusiasts	Avid readers with personal libraries, ages 25-55	Medium	Daily
Casual Readers	Occasional readers looking for specific books	Low to Medium	Weekly
Students	Academic users needing textbooks and references	Medium	Seasonal/High during semester
Community Organizers	Users who facilitate local book clubs and events	Medium to High	Daily

2.3.2 Secondary Users

User Type	Characteristics	Technical Expertise	Usage Frequency
System Administrators	Technical staff managing the platform	High	Daily
Community Moderators	Volunteer users maintaining community standards	Medium	Weekly

2.4 Operating Environment

2.4.1 Hardware Platform

- **Client Side:**
 - Desktop/Laptop computers with modern browsers
 - Mobile devices (responsive web design)
 - Minimum 2GB RAM, 1GB storage space
- **Server Side:**
 - Cloud-based infrastructure (compatible with Neon deployment)

- Auto-scaling capabilities
- CDN for content delivery

2.4.2 Software Platform

- **Client Side:**
 - Web browsers: Chrome 90+, Firefox 88+, Safari 14+, Edge 90+
 - Progressive Web App (PWA) capabilities
- **Server Side:**
 - Runtime: Node.js 18+ LTS
 - Framework: Express.js 4.18+
 - Database: Neon (Serverless PostgreSQL)
 - Version Control: GitHub
 - CI/CD: GitHub Actions
 - Web Server: Nginx (reverse proxy)

2.5 Design and Implementation Constraints

2.5.1 Technology Constraints

- **Frontend Framework:** Angular 17+ with TypeScript
- **Backend Framework:** Node.js with Express.js
- **Database:** Neon Serverless PostgreSQL
- **Version Control:** GitHub repository management
- **Deployment:** GitHub Actions for CI/CD pipeline
- **API Design:** RESTful architecture with JSON responses

2.5.2 Regulatory Constraints

- GDPR compliance for data protection
- Local privacy laws and regulations
- Intellectual property respect for book information
- Community safety guidelines

2.5.3 Business Constraints

- No monetary transactions within the platform
- Focus on local community interactions

- Open-source approach using GitHub for collaboration

2.6 User Documentation

- User manual and getting started guide
- Video tutorials for key features
- FAQ and troubleshooting guide
- Community guidelines and best practices
- API documentation for developers
- GitHub repository documentation

2.7 Assumptions and Dependencies

2.7.1 Assumptions

- Users have reliable internet connectivity
- Users are willing to share personal book collections
- Local communities exist and are active
- Users will maintain accurate book condition information

2.7.2 Dependencies

- Neon database service availability
- Third-party book information APIs (Google Books, OpenLibrary)
- Geolocation services for proximity matching
- Cloud storage services for media files
- Email/SMS services for notifications
- GitHub service availability for CI/CD
- Angular framework updates and support
- Node.js runtime stability

3. System Features

3.1 User Registration and Authentication

3.1.1 Description and Priority

Priority: High

Users must be able to create accounts, log in, and manage their profiles securely using Angular frontend with Node.js backend authentication system.

3.1.2 Stimulus/Response Sequences

1. User accesses registration page (Angular component)
2. System displays registration form with validation
3. User enters required information
4. Angular sends data to Node.js API endpoint
5. Node.js validates data and stores in Neon database
6. System sends verification email
7. User confirms email and activates account

3.1.3 Functional Requirements

ID	Requirement	Priority
REQ-1.1	System shall provide Angular-based registration form with real-time validation	High
REQ-1.2	Node.js backend shall handle user authentication with JWT tokens	High
REQ-1.3	System shall store user credentials securely in Neon database	High
REQ-1.4	System shall send email verification upon registration	High
REQ-1.5	System shall enforce secure password requirements (8+ chars, special chars)	High
REQ-1.6	System shall allow users to reset forgotten passwords via email	High
REQ-1.7	System shall support optional OAuth integration (Google, GitHub)	Medium
REQ-1.8	Angular frontend shall maintain user session state	Medium
REQ-1.9	System shall provide secure logout functionality	Medium

3.2 Book Catalog Management

3.2.1 Description and Priority

Priority: High

Users can add, edit, and manage their personal book collections using Angular components with Node.js API integration and Neon database storage.

3.2.2 Stimulus/Response Sequences

- 1. User selects "Add Book" option in Angular interface
- 2. System displays book entry form with ISBN scanner integration
- 3. User enters book details or scans ISBN
- 4. Angular sends request to Node.js API
- 5. Node.js fetches book information from external APIs
- 6. System stores book data in Neon database
- 7. Angular updates user interface with new book

3.2.3 Functional Requirements

ID	Requirement	Priority
REQ-2.1	Angular interface shall provide intuitive book addition forms	High
REQ-2.2	Node.js shall integrate with book databases for automatic information retrieval	High
REQ-2.3	System shall support ISBN barcode scanning via web camera API	High
REQ-2.4	Neon database shall store book details with proper indexing	High
REQ-2.5	System shall allow users to set book condition (New, Good, Fair, Poor)	High
REQ-2.6	System shall allow users to set sharing status (Available, Lent, Not Available)	High
REQ-2.7	Angular components shall provide real-time book catalog updates	Medium
REQ-2.8	System shall support book categorization by genre and tags	Medium
REQ-2.9	System shall allow users to upload custom book cover images	Medium
REQ-2.10	Node.js shall track book lending history in Neon database	Medium

3.3 Book Discovery and Search

3.3.1 Description and Priority

Priority: High

Users can search for books within their local community using Angular-based search interface with Node.js backend processing and Neon database queries.

3.3.2 Functional Requirements

ID	Requirement	Priority
REQ-3.1	Angular shall provide responsive search interface with auto-complete	High
REQ-3.2	Node.js shall implement full-text search across titles, authors, and genres	High
REQ-3.3	System shall filter search results by availability status	High
REQ-3.4	Angular shall display search results with book details and owner information	High
REQ-3.5	Node.js shall optimize database queries for fast search responses	High
REQ-3.6	System shall sort results by relevance, distance, and availability	High
REQ-3.7	Angular shall provide advanced filters (genre, condition, location radius)	Medium
REQ-3.8	System shall show books within user-defined geographic radius	Medium
REQ-3.9	Node.js shall implement search result pagination for performance	Medium
REQ-3.10	System shall support saved searches with email alerts	Low

3.4 Book Request and Sharing System

3.4.1 Description and Priority

Priority: High

Core functionality enabling users to request books from others and respond to sharing requests through Angular interface and Node.js backend processing.

3.4.2 Functional Requirements

ID	Requirement	Priority
REQ-4.1	Angular shall provide intuitive book request interface	High
REQ-4.2	Node.js shall handle request processing and notifications	High

REQ-4.3	System shall notify book owners of incoming requests via email and in-app	High
REQ-4.4	Angular shall allow owners to approve/decline requests with messages	High
REQ-4.5	Neon database shall track request status (Pending, Approved, Declined, Completed)	High
REQ-4.6	Node.js shall facilitate messaging between requestor and owner	High
REQ-4.7	System shall allow users to set borrowing duration preferences	Medium
REQ-4.8	Angular shall provide request management dashboard	Medium
REQ-4.9	System shall send automated reminders for book return dates	Medium
REQ-4.10	Node.js shall support request queue management for popular books	Low

3.5 AI-Powered Book Recommendation Engine (Novel Feature)

3.5.1 Description and Priority

Priority: High

An intelligent recommendation system built with Node.js machine learning algorithms and Angular visualization components that analyzes user reading preferences, community trends, and book characteristics to suggest relevant books available in the local community.

3.5.2 Key Innovation Points

- **Community-Aware Recommendations:** Unlike commercial platforms that recommend books to buy, this system recommends books available for sharing in the user's local network
- **Angular-Powered Visualization:** Interactive recommendation interface with visual preference mapping
- **Node.js ML Integration:** Server-side machine learning algorithms for pattern analysis
- **Neon Database Analytics:** Advanced queries for community trend analysis
- **GitHub-Tracked Learning:** Version-controlled recommendation algorithm improvements

3.5.3 Technical Implementation

- **Frontend:** Angular components for recommendation display and user feedback
- **Backend:** Node.js with TensorFlow.js or similar ML library

- **Database:** Neon database with optimized queries for recommendation algorithms
- **Analytics:** Real-time user behavior tracking and analysis

3.5.4 Functional Requirements

ID	Requirement	Priority
REQ-5.1	Node.js shall implement machine learning algorithms for user preference analysis	High
REQ-5.2	System shall recommend books available within user's geographic area	High
REQ-5.3	Angular shall provide personalized daily/weekly recommendation dashboard	High
REQ-5.4	Node.js shall use collaborative filtering to find users with similar tastes	High
REQ-5.5	System shall factor book ratings and reviews into recommendation scoring	High
REQ-5.6	Neon database shall store user interaction data for learning algorithm	High
REQ-5.7	Angular shall provide explanation interface for why each book was recommended	Medium
REQ-5.8	Node.js shall identify trending books within local communities	Medium
REQ-5.9	System shall suggest books based on current events and seasonal relevance	Medium
REQ-5.10	Angular shall provide recommendation preferences customization	Medium
REQ-5.11	Node.js shall implement A/B testing for recommendation algorithm improvements	Medium
REQ-5.12	System shall recommend books from underrepresented genres to diversify reading	Low
REQ-5.13	GitHub shall track recommendation algorithm versions and improvements	Low

3.6 Community Features

3.6.1 Description and Priority

Priority: Medium

Features that build and strengthen local reading communities through Angular-based social interfaces and Node.js community management backend.

3.6.2 Functional Requirements

ID	Requirement	Priority
REQ-6.1	Angular shall provide location-based community discovery interface	Medium

REQ-6.2	System shall support community event creation and management	Medium
REQ-6.3	Node.js shall handle community discussion forum backend	Medium
REQ-6.4	Angular shall allow users to follow other community members	Medium
REQ-6.5	System shall display community statistics and leaderboards	Low
REQ-6.6	System shall support book club creation and management	Low
REQ-6.7	Node.js shall implement community moderation tools	Low

3.7 User Communication System

3.7.1 Description and Priority

Priority: High

Built-in messaging system using Angular real-time components and Node.js WebSocket integration for secure communication between users.

3.7.2 Functional Requirements

ID	Requirement	Priority
REQ-7.1	Angular shall provide real-time messaging interface	High
REQ-7.2	Node.js shall implement WebSocket connections for instant messaging	High
REQ-7.3	System shall organize message threads by book requests	High
REQ-7.4	Neon database shall store message history securely	High
REQ-7.5	Angular shall provide message notification system	Medium
REQ-7.6	Node.js shall allow users to share contact information selectively	Medium
REQ-7.7	System shall support message search and filtering	Low

3.8 Rating and Review System

3.8.1 Description and Priority

Priority: Medium

Users can rate and review both books and their experiences with other community members using Angular forms and Node.js processing.

3.8.2 Functional Requirements

ID	Requirement	Priority
REQ-8.1	Angular shall provide interactive rating interface (1-5 stars)	Medium
REQ-8.2	System shall allow users to write and edit book reviews	Medium
REQ-8.3	Node.js shall calculate and store average ratings in Neon database	Medium
REQ-8.4	Angular shall display rating summaries and review highlights	Medium
REQ-8.5	System shall allow users to rate sharing experiences	Medium
REQ-8.6	Node.js shall implement review moderation and reporting system	Low

4. External Interface Requirements

4.1 User Interfaces

4.1.1 Angular Frontend Requirements

- **Framework:** Angular 17+ with TypeScript
- **Responsive Design:** Bootstrap or Angular Material for mobile-first design
- **Component Architecture:** Modular, reusable Angular components
- **State Management:** NgRx for complex state management
- **Routing:** Angular Router for single-page application navigation
- **Progressive Web App:** PWA capabilities for offline functionality

4.1.2 Key Interface Components

Interface	Angular Implementation	Priority
Dashboard	Main component with recommendation widgets and activity feed	High
Book Catalog	Data table component with sorting, filtering, and pagination	High
Search Interface	Search component with auto-complete and advanced filters	High
Request Management	Form components with real-time status updates	High

Community Hub	Social components with discussion threads and events	Medium
Profile Management	Reactive forms for user settings and preferences	Medium

4.2 Hardware Interfaces

4.2.1 Web Browser Integration

- Camera access for barcode/ISBN scanning via WebRTC API
- Geolocation API for location-based features
- Local Storage API for offline functionality
- Push Notification API for browser notifications

4.2.2 Mobile Web Integration

- Responsive design for touch interfaces
- Progressive Web App installability
- Offline-first approach with service workers
- Touch gesture support for mobile interactions

4.3 Software Interfaces

4.3.1 Backend API Interfaces

Component	Technology	Purpose	Priority
Authentication API	Node.js + JWT	User login/registration	High
Book Management API	Node.js + Express	CRUD operations for books	High
Search API	Node.js + Full-text search	Book discovery and filtering	High
Recommendation API	Node.js + ML libraries	AI-powered suggestions	High
Communication API	Node.js + WebSockets	Real-time messaging	High
Community API	Node.js + Express	Social features and events	Medium

4.3.2 External APIs

Service	Integration Method	Purpose	Priority
Google Books API	Node.js HTTP client	Book information retrieval	High
OpenLibrary API	Node.js HTTP client	Alternative book data source	Medium
Google Maps API	Angular Google Maps	Location services and mapping	High
Email Service	Node.js email libraries	Notification delivery	High
GitHub API	Git hooks and CI/CD	Version control and deployment	High

4.3.3 Database Interfaces

- **Primary Database:** Neon (Serverless PostgreSQL)
- **Connection:** Node.js with PostgreSQL client library
- **ORM:** Prisma or TypeORM for database operations
- **Migrations:** Database schema versioning with GitHub tracking
- **Backup:** Automated backup through Neon service

4.4 Communication Interfaces

4.4.1 Network Protocols

- **HTTPS:** All client-server communication
- **WebSocket:** Real-time messaging and notifications
- **REST API:** Standard CRUD operations with JSON
- **GraphQL:** Complex data queries (optional enhancement)

4.4.2 Data Exchange Formats

- **JSON:** Primary API communication format
- **TypeScript Interfaces:** Type-safe data models

- **CSV:** Data import/export functionality
- **PDF:** Report generation for book lists

5. Non-Functional Requirements

5.1 Performance Requirements

5.1.1 Angular Frontend Performance

Metric	Target	Priority
Initial Page Load	< 3 seconds	High
Component Rendering	< 500ms	High
Route Navigation	< 200ms	High
Search Auto-complete	< 300ms	High
Bundle Size	< 2MB (initial)	Medium
Lighthouse Score	> 90 (Performance)	Medium

5.1.2 Node.js Backend Performance

Operation	Response Time Target	Priority
API Authentication	< 200ms	High
Book Search Query	< 500ms	High
Database CRUD Operations	< 300ms	High
Recommendation Generation	< 2 seconds	Medium
File Upload Processing	< 5 seconds	Medium

WebSocket Message Delivery	< 100ms	High
----------------------------	---------	------

5.1.3 Database Performance (Neon)

- **Query Response Time:** < 100ms for indexed queries
- **Connection Pooling:** Efficient connection management
- **Concurrent Connections:** Support 1000+ simultaneous connections
- **Auto-scaling:** Serverless scaling based on demand

5.2 Scalability Requirements

5.2.1 Application Scaling

- **Horizontal Scaling:** Node.js cluster mode support
- **Load Balancing:** Nginx reverse proxy configuration
- **CDN Integration:** Static asset delivery optimization
- **Microservices Ready:** Modular architecture for service separation

5.2.2 Database Scaling

- **Neon Auto-scaling:** Automatic resource allocation
- **Read Replicas:** Database read scaling capability
- **Connection Pooling:** Optimized database connections
- **Query Optimization:** Indexed searches and efficient queries

5.3 Security Requirements

5.3.1 Frontend Security (Angular)

- **Content Security Policy:** XSS attack prevention
- **HTTPS Enforcement:** Secure data transmission
- **Input Validation:** Client-side form validation
- **Authentication Guards:** Route protection with JWT tokens
- **Secure Storage:** Sensitive data protection in browser

5.3.2 Backend Security (Node.js)

- **JWT Authentication:** Secure token-based authentication
- **Input Sanitization:** SQL injection and XSS prevention
- **Rate Limiting:** API abuse protection
- **CORS Configuration:** Cross-origin request security
- **Environment Variables:** Secure configuration management

5.3.3 Database Security (Neon)

- **Encryption at Rest:** Data encryption in Neon database
- **SSL Connections:** Encrypted database communications
- **Access Control:** Role-based database permissions
- **Backup Encryption:** Secure backup storage
- **Audit Logging:** Database access monitoring

5.4 Reliability and Availability

5.4.1 System Availability

- **Uptime Target:** 99.5% availability
- **Neon Reliability:** Leveraging serverless database reliability
- **GitHub Actions:** Reliable CI/CD pipeline
- **Error Handling:** Graceful error recovery in Angular and Node.js
- **Health Monitoring:** Application and database health checks

5.4.2 Disaster Recovery

- **Automated Backups:** Daily database backups through Neon
- **Version Control:** Complete codebase backup on GitHub
- **Deployment Rollback:** Quick rollback capability via GitHub Actions
- **Data Recovery:** Point-in-time recovery with Neon

5.5 Maintainability Requirements

5.5.1 Code Quality

- **TypeScript:** Strong typing for Angular and Node.js
- **ESLint/Prettier:** Code formatting and quality standards
- **Unit Testing:** Jest for Node.js, Jasmine/Karma for Angular
- **E2E Testing:** Cypress or Protractor for integration testing
- **Code Coverage:** Minimum 80% test coverage

5.5.2 Documentation and Versioning

- **GitHub Documentation:** Comprehensive README and wiki
- **API Documentation:** OpenAPI/Swagger specifications
- **Code Comments:** Inline documentation for complex logic
- **Version Control:** Semantic versioning with GitHub releases
- **Change Logs:** Detailed release notes and migration guides

6. Other Requirements

6.1 Development and Deployment Requirements

6.1.1 Development Environment

- **IDE:** Visual Studio Code with Angular and Node.js extensions
- **Package Management:** npm/yarn for dependency management
- **Development Server:** Angular CLI dev server and Node.js nodemon
- **Database Development:** Local Neon development environment
- **Version Control:** Git with GitHub repository

6.1.2 CI/CD Pipeline (GitHub Actions)

- **Build Process:** Automated Angular build and Node.js compilation
- **Testing Pipeline:** Unit tests, integration tests, and E2E tests
- **Code Quality:** ESLint, Prettier, and security scanning

- **Deployment:** Automated deployment to staging and production
- **Database Migrations:** Automated schema updates via Neon

6.1.3 Monitoring and Analytics

- **Application Monitoring:** Performance and error tracking
- **Database Monitoring:** Neon database performance metrics
- **User Analytics:** Privacy-compliant usage analytics
- **GitHub Insights:** Repository and development metrics

6.2 Legal and Compliance Requirements

6.2.1 Data Protection

- **GDPR Compliance:** European data protection regulation
- **CCPA Compliance:** California consumer privacy act
- **Data Retention:** Clear policies for data storage and deletion
- **User Consent:** Transparent consent management system

6.2.2 Open Source Compliance

- **GitHub Public Repository:** Open-source development approach
- **License Management:** Clear licensing for all dependencies
- **Contribution Guidelines:** Community contribution standards
- **Code of Conduct:** Professional development environment

6.3 Accessibility and Internationalization

6.3.1 Accessibility (Angular)

- **WCAG 2.1 Level AA:** Web accessibility compliance
- **Angular CDK A11y:** Accessibility utilities and tools
- **Screen Reader Support:** Proper ARIA labels and roles
- **Keyboard Navigation:** Full keyboard accessibility
- **Color Contrast:** Sufficient contrast ratios

6.3.2 Internationalization (i18n)

- **Angular i18n:** Built-in internationalization support
- **Multi-language Support:** English, Spanish, French (initial)
- **RTL Support:** Right-to-left language preparation
- **Localization:** Cultural adaptation for different regions

6.4 Support and Community

6.4.1 User Support

- **GitHub Issues:** User bug reports and feature requests
- **Documentation Site:** Comprehensive user and developer guides
- **Community Forum:** User-to-user support and discussions
- **Email Support:** Technical support contact system

6.4.2 Developer Community

- **GitHub Contributions:** Open-source contribution guidelines
- **API Documentation:** Developer resources and examples
- **Plugin Architecture:** Extensibility for third-party developers
- **Developer Forum:** Technical discussions and Q&A

Technology Stack Summary

Frontend (Angular)

- **Framework:** Angular 17+ with TypeScript
- **UI Library:** Angular Material or Bootstrap
- **State Management:** NgRx (for complex state)
- **Testing:** Jasmine, Karma, Cypress
- **Build Tools:** Angular CLI, Webpack

Backend (Node.js)

- **Runtime:** Node.js 18+ LTS
- **Framework:** Express.js
- **Language:** TypeScript
- **Authentication:** JWT with Passport.js
- **Testing:** Jest, Supertest
- **API Documentation:** Swagger/OpenAPI

Database (Neon)

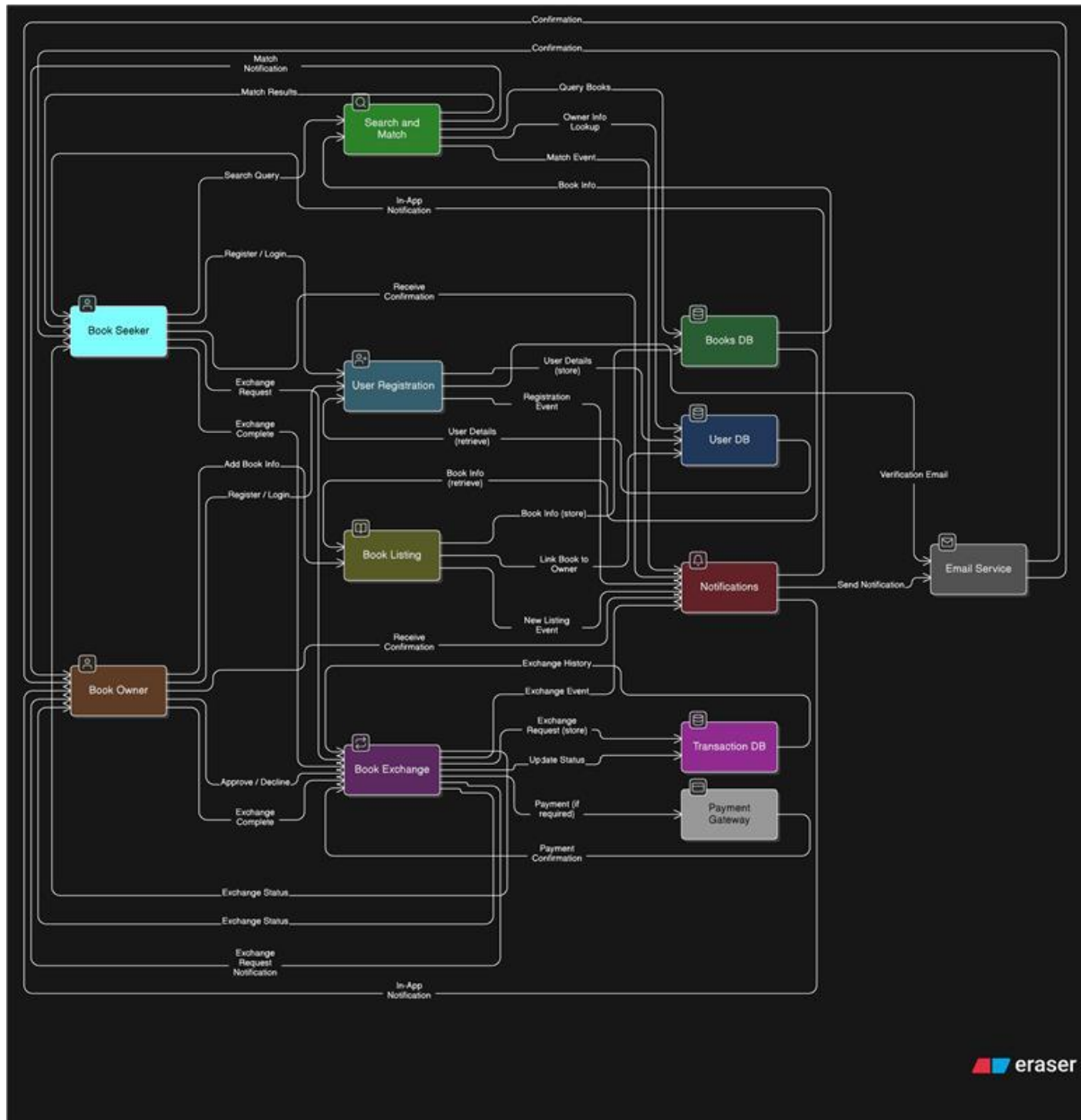
- **Type:** Serverless PostgreSQL
- **ORM:** Prisma or TypeORM
- **Migrations:** Database schema versioning
- **Backup:** Automated through Neon service
- **Scaling:** Auto-scaling serverless architecture

DevOps (GitHub)

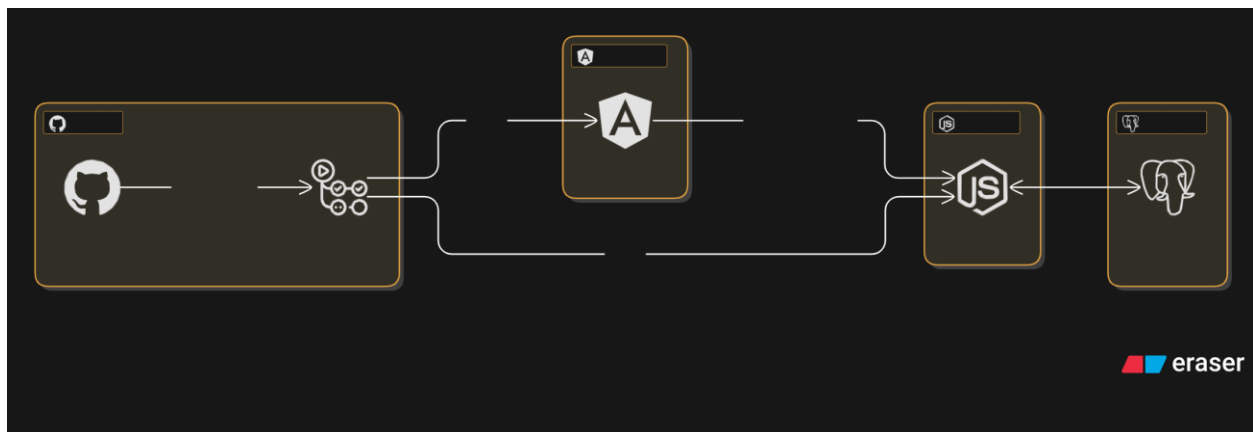
- **Version Control:** GitHub repository
- **CI/CD:** GitHub Actions
- **Issue Tracking:** GitHub Issues
- **Documentation:** GitHub Wiki/Pages
- **Release Management:** GitHub Releases

7. System Diagrams

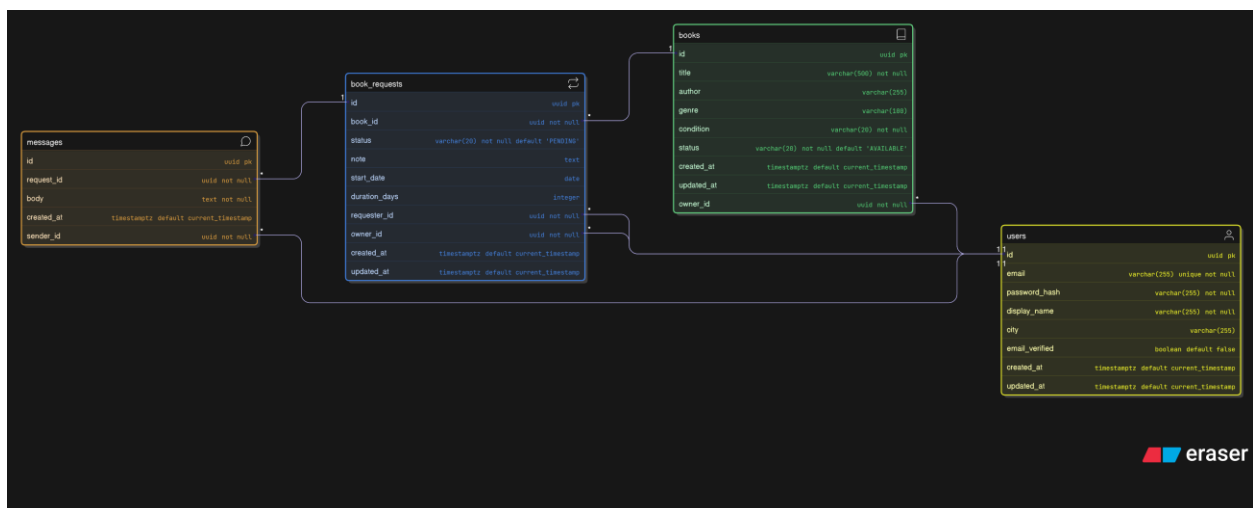
7.1 Data Flow Diagram (DFD)



7.2 System Architecture Diagram



7.3 ER Diagram



Conclusion

This Software Requirements Specification provides a comprehensive foundation for developing the ReadLoop platform using Angular, Node.js, Neon database, and GitHub for development workflow. The document outlines all functional and non-functional requirements necessary to create a robust, scalable, and community-focused book sharing platform.

Key Success Factors:

- **AI-Powered Recommendations:** Intelligent community-aware book discovery
- **Modern Tech Stack:** Angular + Node.js + Neon for scalable performance
- **Open Development:** GitHub-based development for community contributions
- **Serverless Architecture:** Neon database for automatic scaling and reliability
- **TypeScript Throughout:** Type safety across frontend and backend

Technical Advantages:

- Angular's component architecture for maintainable UI
- Node.js performance for real-time features
- Neon's serverless scaling for growth management
- GitHub's collaboration tools for community development
- Full TypeScript implementation for code quality

Next Steps:

1. GitHub repository setup and initial project structure
2. Angular application scaffolding with Angular CLI
3. Node.js API development and Neon database schema design
4. CI/CD pipeline configuration with GitHub Actions
5. Community beta testing and feedback integration

Document Version History:

Version	Date	Changes	Author
1.0	September 1, 2025	Initial document creation with Angular/Node.js/Neon/GitHub stack	Development Team

